Create a language model using Recurrent Neural Network (RNN) on a small corpus of text documents

To create a word-level language model using a Simple Recurrent Neural Network and a small text corpus.

Algorithm
1. Import TensorFlow, NumPy and the required preprocessing libraries.
2. Define a small text corpus directly inside the Python program.
3. Create a tokenizer and fit it on the text corpus.
4. Convert the text into integer token sequences.
5. Generate training sequences using previous words.
6. Pad the sequences to maintain equal input lengths.
7. Separate the input sequences and next-word target labels.
8. Create an Embedding layer to convert words into numerical vectors.
9. Add a SimpleRNN layer to learn sequential information.
10. Add a Dense softmax layer to predict the next word.
11. Compile and train the RNN model.
12. Predict the next word for a sample input sentence.

Program

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras import layers, models
tf.keras.utils.set_random_seed(22)
# Small text corpus
corpus = [
"deep learning is a part of machine learning",
"machine learning uses data to learn patterns",
"deep learning uses neural networks",
"neural networks learn useful patterns from data",
"recurrent networks process sequences of words"
]
# Tokenization
tokenizer = Tokenizer()
tokenizer.fit_on_texts(corpus)
sequences = tokenizer.texts_to_sequences(corpus)
vocab_size = len(tokenizer.word_index) + 1
# Create input sequences
training_sequences = []
for seq in sequences:
for i in range(1, len(seq)):
training_sequences.append(seq[:i + 1])
# Padding
max_len = max(len(s) for s in training_sequences)
training_sequences = pad_sequences(
training_sequences,
maxlen=max_len,
padding='pre'
)
X = training_sequences[:, :-1]
y = training_sequences[:, -1]
# RNN model
model = models.Sequential([
layers.Embedding(vocab_size, 32),

layers.SimpleRNN(64),
layers.Dense(vocab_size, activation='softmax')
])
# Compile
model.compile(
optimizer='adam',
loss='sparse_categorical_crossentropy',
metrics=['accuracy']
)
# Train
model.fit(X, y, epochs=250, verbose=0)
```

```
# Predict next word
seed_text = "deep learning"
seed = tokenizer.texts_to_sequences([seed_text])
seed = pad_sequences(
seed,
maxlen=max_len - 1,
padding='pre'
)
prediction = model.predict(seed, verbose=0)
next_id = int(np.argmax(prediction[0]))
index_word = {
v: k for k, v in tokenizer.word_index.items()
}
print("Input Text:", seed_text)
print("Predicted Next Word:", index_word.get(next_id, "Unknown"))
```

```
Output of Program
Input Text: deep learning
Predicted Next Word: learning-dependent word from corpus
```

Result

Thus, a word-level language model was successfully implemented using a SimpleRNN. The model learned word-sequence patterns and predicted the next word from the given input.